



Week 1 Session 2

Intro to Python

Today's Aims:

- be confident using `print()` and `input()` in python
- understand the different data types in python
- be able to convert between data types as appropriate
- use some basic string methods
- use arithmetic operators

‘I’ve already done this’

If you have studied Python before, you should still find these sessions useful – we need to start with the basics so everyone can catch up, but we will do things in a different way than you may have learned.

There are some advanced tasks and other things for you to be doing while we cover the basics – check the ‘additional tasks’ folder in the repository.

Python

- High level programming language - meaning more like natural (spoken) language than machine code
- Used for a wide variety of different things - particularly data science, natural language processing, and AI
- Easy to work with - gives easy-to-read error messages and has a nice, simple syntax.



Programming in Codespaces

There are two ways of getting back to the repository which you claimed last sessions:

- Open the link from Minerva and click on your email address
- Go to Github and find it in your repositories.



Hello World

It's a programming tradition to write some code which prints out 'Hello World' as your first program.

Open `1_hello_world.py`

Let's write it together.

Variables

A variable is a space where you can store some data.

We give it a **variable name** and an initial **value**.

The value can **change** throughout the program.

```
name = "Amy"
```

```
date = "20-10-2024"
```

```
speed = 11.423
```

```
distance = 102374
```

Variables Continued

There are some rules about variable names:

- no spaces
- no symbols
- always start with a letter
- don't use **reserved** words (names of existing functions, for example 'input' or 'print')

In python, we usually use snake_case to write variable names - where you would have a space, use an **_underscore_** instead.

Printing With Variables

You can print the value of a variable very easily:

```
print(name)
```

And we can create **f-strings** which contain variables and text:

```
print(f"Hello {name}")
```

Task 1

- Add to your 'hello world' so that it prints out a name stored in a variable
- Make your program say hello to three different people by changing the value and printing again.

User Input

Instead of **hard-coding** variables - having their value set in the code - we can get users to input values:

```
name = input("What is your name?")
```

Let's adapt our code to do this!

Task 2

- Adapt your 'hello world' program to have a conversation
 - Ask 3 questions, and reply to your user in a way which uses their response
 - For example:

```
What's your name? Amy
Oh cool, I had a friend at school called Amy
Where did you grow up? York
I hear there's a nice museum in York!
What's your favourite animal? echidnas
I personally prefer giraffes but echidnas are cool too I guess
```
- You can use your own questions and responses!



Different Types of Variable

Integers, strings and floats

Numbers

Now we can get numbers into the program, let's make a very basic calculator.

2_calculator.py

- Get a user to enter two numbers
- Make another variable 'answer' which holds the result of adding them together (hint: use +)
- Print out the answer for the user

Why didn't that work?

When we use `input()` Python stores the value as a data type called a **string**.

Python doesn't recognise that the numbers are numbers - it is treating them as text.

We need to use a different **data type**.

We will look at 3 basic data types today:

Data Types



String - any text, symbols or numbers inside of inverted commas
("hello", "3.83.11", "01904")



Integer - whole numbers
(3, 974, -1)



Float- numbers with decimal points
(3.1415, -8.624, 3.0)

Fixing Our Calculator

To turn our strings into integers, we will be converting them.

The easiest way to do this is to do this when we read the input:

```
num = int(input("Enter a number: "))
```

Task 3 – Common Errors

It's easy to make mistakes when programming – and it's important to start to understand how to find and fix them.

3_debugging.py contains several common errors I see students make.

You need to find and fix them all so that the code works.

User Error

- Try entering a letter instead of a number in any input which has an `int()` around it...

```
Enter your first number: no
Traceback (most recent call last):
  File "/workspaces/the-junk-drawer/example2.py", line 1, in <module>
    num = int(input("Enter your first number: "))
           ~~~~~^
ValueError: invalid literal for int() with base 10: 'no'
```

try and
except
function

Writing Robust Code

As a user, you do not want the program you are using to do this.

As a user, you want programs to **tell you what you did wrong and let you try again!**

As a developer, it's your job to **make things fail gracefully** – this means we should think about:

- What might our user do wrong?
- What should the program do in response?
- How do we test that this happens?

Try... Except...

One of the best ways to stop errors reaching your users is to use try ... except

```
try:
```

```
    num1 = int(input("Enter your number: "))
```

```
    num2 = int(input("Enter your number: "))
```

```
    answer = num1 + num2
```

```
    print(f"{num1} + {num2} = {answer}")
```

```
except:
```

```
    print("Please enter numbers only.")
```

This will prevent the program from continuing if there is an error.

Try... Except... Explained

What does it mean?

try: Run this code until it ends or there is an error

```
num1 = int(input("Enter your number: "))
```

```
num2 = int(input("Enter your number: "))
```

```
answer = num1 + num2
```

```
print(f"{num1} + {num2} = {answer}")
```

except: And if it causes an error, do this instead

```
print("Please enter numbers only.")
```



Maths in Python

Arithmetic Operators

Basic Arithmetic

Like other programming languages, Python can do maths.

You might recognise the basic operators:

+

-

*

/

**

Let's Get Some Practice!

For each of these maths expressions, calculate the answer using Python.
You can hard-code the values.

4_maths_practice.py

1. $(4 \times 8) \times 6$
2. $(2^3) / (8/3)$
3. $27^2 \times 19/4$

Hint: ^ doesn't work in python

Formatting Float Outputs

You may have noticed that python will print out many decimal places when working with floats.

This is often unhelpful when writing programs and makes it harder to read answers.

We can use f-strings to tell python to print a sensible amount:

print three decimal points

```
print(f"The answer is {answer:.3f}")
```

Modulo and Integer Division

In programming, we have two more unusual operators:

// - integer division – gives you only the **whole** part of a division

$$8 \ // \ 3 = 2$$

% - modulo division – gives you only the **remainder** of a division

$$9 \ // \ 2 = 1$$

Using Modulo and Integer Division

These are very useful for programming although they aren't commonly seen outside of computer science.

Modulo is used for a wide array of things, for example deciding whether a number is even or odd.

Let's look at how we can use them together on a real problem:



Modulo Task

5_module_intdiv.py

If you're feeling confident, you can start working on the task individually.

I will talk through it for anyone who wants some help.

Strings

One of the things which makes Python a very useful and popular programming language is how easy it makes it to work with **strings**.

It has a huge variety of **methods** (special functions built into a data type) which can do some quite impressive things with strings in a single command.

We're not going to go through all of them together – instead, you're going to use some of them yourself.

What can they do?

In '6_strings.py' there are lots of different string methods - run the code and work out what each one does. Write a #comment to explain what it does.

Gradescope & Portfolios

You will be submitting your weekly portfolio tasks to Gradescope.

Sometimes these will be automatically graded by a script – for this to work, you need to upload the code with the right name and format.

To practice, you are going to complete a very short task and upload it to Gradescope to make sure you understand this process.

`7_gradescope.py`

End of Session Reminder

You need to **add, commit and push** your work to save it!

```
git add -A
```

```
git commit -m "end of session 2"
```

```
git push
```